МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки»

Лабораторная работа №5
по курсу «Программирование графических процессоров»

Сортировка чисел на GPU. Свертка, сканирование, гистограмма. Битоническая сортировка (вариант 1)

Выполнил: Голев Андрей Михайлович
Группа: М8О-401Б-23
Преподаватель: А.Ю. Морозов

Москва, 2026

Условие

Цель работы - ознакомление с фундаментальными алгоритмами GPU: сверткой (reduce), сканированием (blelloch scan) и гистограммой (histogram); реализация одной из сортировок на CUDA; использование разделяемой и других видов памяти; исследование производительности программы с помощью утилиты nvprof. Все входные и выходные данные - бинарные: программа читает их из стандартного ввода и пишет в стандартный вывод.

Мой вариант - №1, битоническая сортировка. Требуется реализовать битоническую сортировку для чисел типа int с адаптивной операцией битонического слияния: если данные помещаются в разделяемую память, взаимодействие идёт через неё, если нет - через глобальную память (то есть необходимо реализовать несколько вариантов ядра). Ограничение: $n \le 256*10^6$. Вход: в первых четырёх байтах записано целое число n - длина массива, далее следуют n чисел типа int. Выход - n отсортированных по возрастанию чисел в бинарном виде. Пример из условия: десять чисел от 0 до 9, поданные в порядке 0, 9, 8, …, 1, на выходе дают 0, 1, …, 9.

Программное и аппаратное обеспечение

Код я писал, отлаживал и компилировал на виртуальной машине с GPU, на ней же проводил все замеры времени и профилировку.

Графический процессор - NVIDIA GeForce GTX 1650: compute capability 7.5, 14 мультипроцессоров, глобальная память 4 086 169 600 байт (примерно 3,8 ГБ), разделяемая память на блок 48 КБ, константная память 64 КБ, 65 536 регистров на блок, максимум 1024 нити на блок, максимальная сетка (2 147 483 647, 65 535, 65 535), размер варпа 32.

Из софта: Ubuntu 24.04.1 LTS, драйвер NVIDIA 550.144.03, CUDA 12.4, компилятор nvcc 12.0 (V12.0.140), g++ 13.3.0. Боевая программа компилируется командой из условия: nvcc -Werror cross-execution-space-call -lm. Для замеров я написал отдельный бенчмарк bench5.cu (компилируется тем же nvcc с -O2; CPU-эталон в нём - последовательный std::sort), генератор бинарных входных данных gen_input.py, скрипт сквозной проверки verify.py и скрипт run_bench.sh, который компилирует всё, снимает параметры GPU, прогоняет nvprof на боевой программе и бенчмарк и собирает таблицы для отчёта.

Метод решения

Битонической называется последовательность, которая сначала монотонно растёт, а затем монотонно убывает (или циклический сдвиг такой последовательности). Сортировка Бэтчера состоит из фаз k = 2, 4, …, m, где m - размер массива, степень двойки; в фазе k выполняются стадии j = k/2, k/4, …, 1. На стадии j каждый индекс i участвует в сравнении-обмене (compare-and-swap) с партнёром i ^ j (побитовое исключающее ИЛИ; j - степень двойки, поэтому партнёр отличается от i ровно одним битом); направление обмена задаётся битом (i & k): если он нулевой, пара упорядочивается по возрастанию, иначе - по убыванию. Фазы $k < m$ постепенно строят из входа битонические последовательности длины k, а фаза k = m - финальное слияние: направление всех её обменов восходящее ((i & m) = 0 для всех $i < m$), и после неё весь массив отсортирован по возрастанию. Всего $\log_2 m*(\log_2(m+1))/2$ стадий, каждая - n/2 независимых сравнений-обменов; последовательность операций не зависит от данных (алгоритм oblivious), поэтому между стадиями нет зависимостей, требующих участия хоста.

Битоническая сортировка требует массив длины m = 2^round($\log_2 n$). Исходный массив дополняется до m значением INT_MAX: после сортировки все элементы дополнения оказываются в конце, и выводятся первые n элементов - результат совпадает с сортировкой исходного массива, даже если во входе встречается INT_MAX. Хвост [n, m) заполняется на GPU отдельным ядром: cudaMemset не подходит, так как байтовый шаблон 0xFF задаёт значение -1, а не INT_MAX.

Адаптивное битоническое слияние устроено так. На стадии (k, j) обмены происходят внутри выровненного чанка размером 2*j: партнёры i и i ^ j всегда лежат в одном чанке. Если чанк помещается в разделяемую память блока (2*j ≤ 8192 элементов - 32 КБ при 4 байтах на элементе, с запасом против лимита 48 КБ), слияние идёт через разделяемую память: блок загружает чанк целиком и выполняет в нём все стадии от j до 1 - они все лежат внутри того же чанка, - после чего выгружает чанк обратно. Если чанк не помещается (2*j > 8192), стадия выполняется через глобальную память. Отдельный случай - весь массив: если m ≤ 8192, он целиком помещается в разделяемую память одного блока, и вся сортировка - все фазы и стадии - выполняется одним блоком без выхода в глобальную память.

Итого при m > 8192 каждая фаза k выполняется так: стадии j = k/2, …, 8192 - запусками глобального ядра (по одной стадии на запуск), затем один запуск ядра разделяемой памяти, которое по чанкам из 8192 элементов выполняет все стадии j = min(k/2, 4096), …, 1. Для m = 2^s это s запусков merge-ядра и (s-13)(s-12)/2 запусков глобального ядра; на предельном m = 2^{28} - 28 + 120 = 148 запусков.

Все ядра используют grid-stride циклы с фиксированными конфигурациями запуска, так что число нитей не зависит от n (требование курса). Сортировка идёт in-place в глобальной памяти устройства: на предельном n = 256*10^6 дополненный массив занимает 2^{28}*4 = 1 ГБ из 3,8 ГБ.

Описание программы

Программа - один файл lab5.cu. Четыре ядра.

fillTailKernel(data, from, to) заполняет хвост [from, to) значением INT_MAX: нити идут по диапазону grid-stride циклом. Конфигурация запуска <<<512, 256>>> - 131 072 нити.

bitonicStageGlobal(data, m, k, j) - одна стадия (k, j) через глобальную память: нить grid-stride циклом идёт по индексам i и обрабатывает пару (i, i ^ j), если i ^ j > i, - так каждая пара обрабатывается ровно одной нитью; направление обмена определяется битом (i & k). Рабочая конфигурация <<<512, 256>>>.

bitonicMergeSmem(data, m, k, jstart) - адаптивное слияние через разделяемую память: каждый блок берёт чанк из 8192 элементов (по 8 на нить при 1024 нитях), загружает его в разделяемую память, выполняет стадии j = jstart, …, 1 (после каждой - __syncthreads()) и выгружает чанк; по чанкам - grid-stride цикл. Направление обмена вычисляется по глобальному индексу чанка (base + i), поэтому ядро корректно работает и когда направление меняется внутри чанка (фазы k < 8192), и когда оно постоянно в пределах чанка (k ≥ 8192). Рабочая конфигурация <<<512, 1024>>> - 524 288 нитей.

bitonicSortSmem(data, m) - полная сортировка одним блоком для m ≤ 8192: блок загружает весь массив в разделяемую память и выполняет все фазы и стадии, направление определяется битом (i & k).

fail(msg) печатает «ERROR: …» в stderr и завершается с кодом 0 - по требованию курса программа не должна завершаться аварийно ни при каких условиях.

main читает из stdin первые четыре байта - n (отрицательное или превышающее $256*10^6$ значение - ошибка; n = 0 - пустой вывод), затем n чисел int в закреплённый (pinned) буфер - cudaHostAlloc с fallback на malloc, закреплённая память заметно ускоряет пересылку гигабайтного массива. Вычисляется m - ближайшая степень двойки; на устройстве выделяется m*4 байт, копируются n элементов, ядром fillTailKernel заполняется хвост. Далее при $m \leq 8192$ запускается bitonicSortSmem, иначе для каждой фазы k - запуски bitonicStageGlobal для j = k/2, …, 8192 и один запуск bitonicMergeSmem для остальных стадий. После cudaDeviceSynchronize отсортированные n элементов копируются обратно и выводятся в stdout. Каждый вызов CUDA проверяется, при ошибке - fail.

Результаты

Время мерял событиями CUDA (cudaEventRecord/cudaEventElapsedTime) в бенчмарке bench5.cu: интервал охватывает всю последовательность запусков ядер сортировки, без пересылки данных. В таблице 1 - среднее по 20 запускам для $n = 2^{10}$ и 2^{13}, по 10 для 2^{16} и 2^{18}, по 5 для 2^{20} и 2^{22} и по 3 для 2^{24} и 2^{26}; перед замерами - прогревочные запуски (3, 3, 3, 2, 2, 2, 1, 1 соответственно). Массивы генерируются детерминированным конгруэнтным ГПСЧ (seed 42), значения равномерны по всему диапазону int. CPU-эталон - последовательный std::sort, среднее по 5, 5, 3, 3, 2, 2, 1, 1 запускам. Результат каждой конфигурации сверялся с CPU-эталоном побайтово (memcmp): 9 проверок, все OK. Боевая программа дополнительно проверена сквозно скриптом verify.py на трёх размерах: выход отсортирован по возрастанию и совпадает с отсортированным входом.

Таблица 1 - Время битонической сортировки по размерам (рабочая конфигурация)

Размер n	GPU, мс (event)	GPU, мс (wall)	CPU std::sort, мс	Ускорение	H2D+D2H, мс
1024	0,043	0,052	0,029	0,7	0,140
8192	0,133	0,141	0,311	2,3	0,130
65 536	0,288	0,298	2,863	9,9	0,560
262 144	0,944	0,958	12,955	13,7	1,791
1 048 576	4,887	4,905	56,698	11,6	5,958
4 194 304	22,505	22,529	250,476	11,1	27,019
16 777 216	104,074	104,094	1 101,683	10,6	100,759
67 108 864	525,049	525,079	4 696,074	8,9	408,498

Для $n \leq 8192$ массив целиком помещается в разделяемую память и сортируется одним блоком; остальные размеры идут адаптивным путём. На n = 1024 GPU проигрывает CPU (0,043 против 0,029 мс): массив крошечный, и время определяется запуском ядра, а не полезной работой. С ростом n ускорение растёт до пика 13,7 на $n = 2^{18}$, затем спадает до 8,9× на 2^{26}.

Спад объясняется двумя причинами. Во-первых, битоническая сортировка выполняет $O(n*\log^2 n)$ сравнений против $O(n*\log n)$ у std::sort: лишний множитель $\log_2 m = 26$ на предельном размере съедает преимущество. Во-вторых, растёт пересылка: на 2^{26} копирование H2D+D2H занимает 408 мс - 44% полного времени программы (933 мс с учётом пересылки), и полное ускорение против CPU составляет 5,0× (на 2^{24} - 5,4).

Показательно, что wall-время почти не отличается от event-времени (525,079 против 525,049 мс на 2^{26}): последовательность из 117 запусков ядер выполняется без простоев, синхронизация с хостом одна на весь запуск.

Перебор конфигураций на $n = 2^{22}$: для stage-ядра восемь комбинаций grid × block от <<<64, 128>>> до <<<1024, 256>>> дают 21,185–22,064 мс - зависимость слабая (разброс около 4%), рабочая конфигурация <<<512, 256>>> с 21,201 мс практически лучшая (у <<<1024, 256>>> - 21,185 мс). Для merge-ядра (block 1024 фиксирован - 8 элементов на нить) сетки из 56–896 блоков дают 20,445–21,174 мс; лучшая <<<112, 1024>>> быстрее рабочей <<<512, 1024>>> на 3,7%. Заметим, что в основном проходе та же рабочая конфигурация дала 22,505 мс - на 6% больше, чем в непрерывной серии свипа: это эффект прогрева тактов GPU (перед этим замером программа долго сортировала на CPU и GPU простаивал), а не разброс измерений.

Профилировка боевой программы утилитой nvprof (бинарные входы из случайных чисел): n = 4096 - путь разделяемой памяти, $n = 2^{20}$ и 2^{24} - адаптивный путь. На двух последних размерах fill-ядро не запускается: n - степень двойки, дополнение не требуется.

Таблица 2 - nvprof: разбивка по операциям GPU, боевая программа, n=4096

Операция GPU	**Время, мс**	**Вызовов**	**Доля, %**
Сортировка (bitonicSortSmem)	0,074	1	88,5
Копирование DtoH	0,008	1	9,9
Копирование HtoD	0,001	1	1,6
Вся GPU-активность	0,083	-	100

Таблица 3 - nvprof: разбивка по операциям GPU, боевая программа, $n=2^{20}$

Операция GPU	**Время, мс**	**Вызовов**	**Доля, %**
Слияние в разделяемой памяти (bitonicMergeSmem)	3,333	20	32,7
Копирование HtoD	2,742	1	26,9
Копирование DtoH	2,458	1	24,1
Стадии через глобальную память (bitonicStageGlobal)	1,663	28	16,3
Вся GPU-активность	10,196	-	100

Таблица 4 - nvprof: разбивка по операциям GPU, боевая программа, $n=2^{24}$

Операция GPU	**Время, мс**	**Вызовов**	**Доля, %**
Стадии через глобальную память (bitonicStageGlobal)	60,912	66	31,2
Слияние в разделяемой памяти (bitonicMergeSmem)	50,784	24	26,0
Копирование HtoD	44,204	1	22,6
Копирование DtoH	39,451	1	20,2
Вся GPU-активность	195,351	-	100

Картина по nvprof такая. На n = 4096 вся сортировка - один запуск bitonicSortSmem (0,074 мс, 88,5% GPU-времени): массив целиком в разделяемой памяти, из глобальной памяти - только копирование 16 КБ на вход и выход. На $n = 2^{20}$ - 48 запусков ядер (28 глобальных стадий и 20 слияний): сами ядра занимают 4,996 мс (49,0% GPU-времени), а

пересылка 4 МБ в обе стороны - 5,200 мс (51,0%). На $n = 2^{24}$ - 90 запусков (66 и 24): ядра 111,696 мс (57,2%), пересылка 64 МБ - 83,655 мс (42,8%); в среднем глобальная стадия занимает 0,92 мс, слияние в разделяемой памяти - 2,12 мс. Пересылка DtoH в боевой программе быстрее, чем в бенчмарке на обычной памяти (39,451 против 57,440 мс на том же объёме), - эффект закреплённой памяти. По API-статистике единственный крупный разовый расход - cudaHostAlloc (185,76 мс на закрепление 64 МБ); на 90 запусков ядер cudaLaunchKernel тратит 0,84 мс - накладные расходы на запуск пренебрежимо малы.

Выводы

Битоническая сортировка - oblivious-алгоритм: последовательность сравнений-обменов не зависит от данных, а между стадиями нет зависимостей, требующих участия хоста. Вся сортировка - непрерывная серия запусков ядер (90 на $n = 2^{24}$, 148 на предельном $m = 2^{28}$), и GPU не простаивает: wall-время совпадает с event-временем с точностью до сотых долей процента. Это принципиальный контраст с последовательными по итерациям алгоритмами вроде метода Гаусса из лабораторной работы №4, где каждая итерация требовала синхронизации хост–устройство и GPU простаивал большую часть времени.

Адаптивное слияние сокращает и число запусков ядер, и обращения к глобальной памяти. Стадии с рабочим чанком до 8192 элементов идут через разделяемую память: merge-ядро выполняет все стадии слияния фазы внутри чанка за одну загрузку. На $m = 2^{24}$ это 24 запуска merge-ядра вместо 221 запуска, если бы каждая стадия с $j \leq 4096$ шла отдельным глобальным ядром, - всего 90 запусков вместо 287, а обмены внутри чанков идут через быструю разделяемую память. Случай $m \leq 8192$ вырождается в один запуск одного блока.

Ускорение против последовательного std::sort достигает 13,7 ($n = 2^{18}$) и держится 9–11 на больших размерах; с учётом пересылки данных - около 5. Спад с ростом n предопределён сложностью $O(n*\log^2 n)$ против $O(n*\log n)$ и растущей долей пересылки через PCIe (43% GPU-времени на $n = 2^{24}$).

Возможные оптимизации: перекрыть пересылку с вычислениями (начинать сортировку первых блоков данных до окончания копирования, двойная буферизация на потоках CUDA); сократить накладные расходы на сотню запусков ядер с помощью CUDA Graphs; увеличить число элементов на нить в глобальных стадиях; применять cooperative groups для объединения нескольких длинных стадий в один запуск ядра.